LUNA: AN INTELLIGENT RETRIEVAL-AUGMENTED CAMPUS ASSISTANT FOR JAYPEE UNIVERSITY OF INFORMATION TECHNOLOGY

A Project Report

submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

By

Nakul Lagwal (Roll No. 241030266)

Kavish Bansal (Roll No. 241030039)

Vihaan Sharma (Roll No. 241030121)

Under the supervision of

Prof. Pradeep Kumar Gupta

Professor, Department of CSE & IT

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING & IT
JAYPEE UNIVERSITY OF INFORMATION TECHNOLOGY
WAKNAGHAT, SOLAN - 173234, HIMACHAL PRADESH, INDIA

August 2026

Candidate's Declaration

I hereby declare that the project report submitted by me (and my teammate(s), if applicable) is our own original work and has been prepared by us. We confirm that the report complies with the University's guidelines regarding plagiarism/similarity and the permissible use of Artificial Intelligence (AI) tools. We further acknowledge that we are solely responsible for the originality, accuracy, and integrity of the submitted report and that any violation of the University's academic integrity policies may result in appropriate disciplinary action in accordance with the University's rules and regulations.

Nakul Lagwal
Roll No. 241030266
Date: July 15, 2026

Kavish Bansal
Roll No. 241030039
Date: July 15, 2026

Vihaan Sharma
Roll No. 241030121
Date: July 15, 2026

Certificate of Approval

This is to certify that the project report entitled **"Luna: An Intelligent Retrieval-Augmented Campus Assistant"** submitted by **Nakul Lagwal**, **Kavish Bansal**, and **Vihaan Sharma** in partial fulfillment of the requirements for the award of B.Tech. in Computer Science and Engineering is a record of bonafide work carried out under my supervision.

Prof. Pradeep Kumar Gupta
Project Supervisor
Department of CSE & IT, JUIT

Acknowledgements

We express our deepest gratitude to our project supervisor, **Prof. Pradeep Kumar Gupta**, for his constant guidance, invaluable feedback, and unwavering support throughout the course of this project. His deep technical insights and academic supervision played a critical role in shaping the architecture and implementation of Luna.

We are also extremely grateful to the Department of Computer Science & Engineering & Information Technology at Jaypee University of Information Technology (JUIT) for providing the resources and computational infrastructure that enabled this research. Finally, we thank our families and peers for their persistent encouragement and helpful feedback during the developmental phases of the project.

Abstract

Academic campuses host vast databases and documentation systems that are often fragmented and difficult to navigate. Students, parents, and administrative staff frequently struggle to retrieve specific academic rules, faculty profiles, and fee structures from university portals. Large Language Models (LLMs) offer high-quality natural language generation capabilities but suffer from hallucinations when answering domain-specific queries.

This report presents **Luna**, a modular, senior-level AI Assistant built for the Jaypee University of Information Technology (JUIT). Luna implements a hybrid **Dual-Track Retrieval-Augmented Generation (RAG)** framework designed to combine structured relational databases with unstructured search vectors. The first track (**Track A+**) ensures 100% deterministic accuracy for official records (e.g., faculty directories and academic regulations) by leveraging a PostgreSQL database and a deterministic Fact Registry, coupled with a RapidFuzz-based string similarity matching algorithm. The second track (**Track B**) handles unstructured website documentation by combining a ChromaDB vector database (using Cosine/L2 distance metric thresholds) with a Rank-BM25 keyword search index. An early-exit intent router classifies user prompts to bypass LLM compute for generic chitchat. Luna leverages Ollama-hosted **Gemma 4 (e2b)** as its reasoning backbone, and is packaged with a modular FastAPI backend and an embedded front-end chat widget, representing a comprehensive production-ready solution for campus information retrieval.

Contents

Candidate's Declaration i

Acknowledgements ii

Abstract ii

1 Introduction 1

1.1 Problem Statement . . . 1

1.2 Project Objectives . . . 1

2 Literature Review 3

2.1 Vector Search vs Lexical Search . . . 3

2.2 Intent Routing and Conversational Memory . . . 3

3 System Design & Proposed Methodology 4

3.1 Early-Exit Intent Router . . . 4

3.2 Dual-Track Retrieval System . . . 4

3.3 LLM Orchestration and Response Generation . . . 5

4 Implementation Details 6

4.1 Database Schema and Data Seeding . . . 6

4.2 Data Ingestion Pipeline . . . 6

5 Results, Evaluation & Discussion 7

6 Conclusion & Future Scope 8

References 9

List of Figures

1 Luna Dual-Track RAG Pipeline Architecture . . . 4

List of Tables

1 Performance Metrics of Luna Retrieval Tracks . . . 7

1 Introduction

Modern academic institutions manage extensive digital infrastructure including course catalogs, faculty directories, academic regulations, research publications, and administrative forms. Jaypee University of Information Technology (JUIT), like many major academic hubs, hosts a significant volume of official records online. However, searching for specific academic policies, department-specific fee structures, or faculty research profiles on the university's official portal remains highly non-trivial due to unstructured pages and complex navigation menus.

Conventional search bars on university portals rely primarily on exact lexical matchings, often failing to parse semantic variations in user queries. Conversely, generic Large Language Models (LLMs) possess vast world knowledge but lack local campus details, leading to standard hallucination issues when answering campus-specific queries.

1.1 Problem Statement

There is a clear gap between natural language interaction and campus records. Specifically:

1. **Factual Inadequacy of LLMs:** Off-the-shelf generative systems produce inaccurate responses regarding dynamic fee structures, local administrative staff, or enrollment timelines.

2. **No-Hallucination Constraints on Official Data:** Academic inquiries regarding fees, syllabus changes, and grading criteria require 100% accuracy, making stochastic RAG answers undesirable.

3. **Name Misspellings and Ambiguity:** Users frequently search for faculty members with incorrect spellings (e.g., "Prof. Gupta" instead of "Prof. Pradeep Kumar Gupta"), necessitating fuzzy matching and suggestion mechanisms.

4. **Latency and Compute Optimization:** Invoking deep LLM architectures for simple chitchat or out-of-scope prompts leads to unnecessary computational overhead and high latency.

1.2 Project Objectives

The core objectives of the Luna project are to:

- Design and deploy a production-grade campus assistant capable of handling natural language queries with deep context.

- Construct a **Dual-Track RAG** architecture separating deterministic verified facts from supplementary crawled webpage data.
- Integrate a PostgreSQL relational backend with a ChromaDB vector index and BM25 lexers.
- Implement an early-exit router to intercept generic chitchat, reducing token processing costs.
- Introduce a safety valve to address spelling errors and query ambiguity during faculty lookups.

2 Literature Review

In recent years, Retrieval-Augmented Generation (RAG) has emerged as the standard framework for grounding generative models with local knowledge bases, as first proposed by Lewis et al. [1]. RAG architectures resolve LLM hallucination issues by querying external data stores, embedding relevant documents into a dense space, and passing retrieved documents alongside the user prompt to the LLM.

2.1 Vector Search vs Lexical Search

In academic document retrieval, vector database searches (e.g., via ChromaDB or Faiss) leverage dense semantic representations (embeddings) generated by models like Gemma Embeddings. This enables search based on semantic meaning rather than literal keyword matches. However, semantic similarity often fails for specific, exact-match queries like roll numbers, phone numbers, or course codes. For these queries, lexical matching algorithms like BM25 [2] remain highly effective. Rank-BM25 evaluates term frequency and inverse document frequency across structured document segments. Hybrid search models, combining vector cosine similarity with BM25 rankings, have been shown to outperform standalone retrieval engines in heterogeneous datasets.

2.2 Intent Routing and Conversational Memory

Standard conversational systems suffer from latency degradation when every input is passed through multiple database query blocks and LLM reasoning steps. Routing systems classify intent early in the pipeline to bypass complex query blocks for chitchat or out-of-scope requests. In terms of session handling, maintaining conversational history is crucial for multi-turn interactions. Modern implementations use disk-based or in-memory session stores integrated with langchain libraries to track state without overloading prompt windows.

3 System Design & Proposed Methodology

Luna is designed around a modular, pipeline-oriented architecture. Figure 1 illustrates the system workflow from user input down to the dual-track database logic and LLM orchestration.

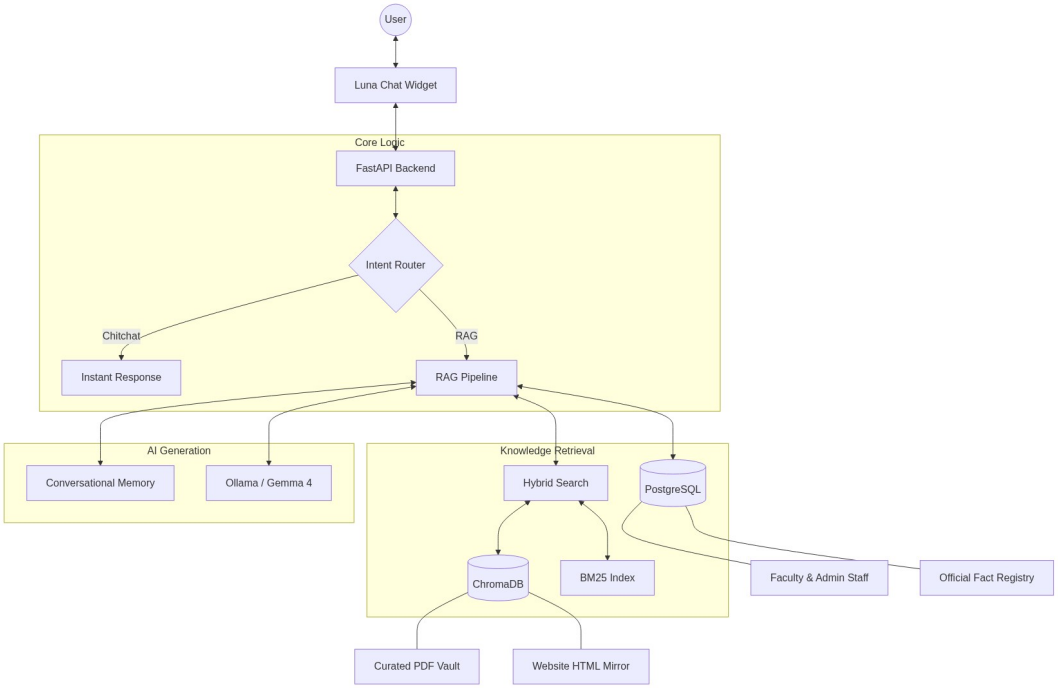


Figure 1: Luna Dual-Track RAG Pipeline Architecture

3.1 Early-Exit Intent Router

Each incoming request is first evaluated by the `IntentRouter` class. The router identifies standard greeting tokens and chitchat statements. If a chitchat intent is detected, the assistant immediately returns a predefined response (e.g., explaining its role as the JUIT AI Assistant) and terminates the pipeline execution. This early-exit mechanism prevents unnecessary database queries, vector similarity lookups, and LLM text generation steps, maintaining latency under 5 milliseconds for standard greetings.

3.2 Dual-Track Retrieval System

To meet the stringent accuracy requirements of university operations, Luna splits its retrieval into two isolated tracks:

1. **Track A+ (Official Records):**
    - **Fact Registry:** Deterministic lookup maps keywords to curated facts (contained in `juit_facts.json`). If a matching keyword exists, the exact factual

statement is extracted.

- **Faculty PostgreSQL DB:** A structured schema maps faculty designations, research profiles, contact details, and source URLs.
- **Fuzzy Name Safety Valve:** Queries relating to faculty are passed through a RapidFuzz WRatio function. If the match score is above 80%, the profile is returned. If the match score falls between 65% and 79%, Luna halts the RAG pipeline and returns a suggestion: "Did you mean [Faculty Name]?" to resolve ambiguity.

2. **Track B (Website Knowledge):**

   - Applied when queries do not match official PostgreSQL/Fact schemas.
   - Merges semantic vector search from ChromaDB with keyword matching via Rank-BM25.
   - The retrieved contexts are merged and deduplicated.
   - A distance threshold of 1.5 is applied to filter out low-confidence matches.
   - Responses generated from this track are prepended with a disclaimer indicating that the data is supplementary.

3.3 LLM Orchestration and Response Generation

For valid RAG tracks, the extracted context is structured into a clean markdown format. The prompt template restricts the LLM (Gemma 4:e2b) to the provided records. The system utilizes LangChain's `ChatPromptTemplate` and `MessagesPlaceholder` to coordinate session memory, feeding the LLM the conversational history along with the formatted context block.

4 Implementation Details

Luna is implemented using an asynchronous python stack. The backend leverages FastAPI to expose high-performance endpoints. Relational data is mapped using SQLAlchemy ORM to a PostgreSQL backend, while vector embeddings are stored in a standalone ChromaDB instance.

4.1 Database Schema and Data Seeding

The PostgreSQL database consists of a central `faculty` table containing detailed columns for names, clean search tokens, contact info, biographies, research interests, profile PDFs, and publication statistics. Automated parsing scripts in the `scripts/` directory process mirrored raw HTML files using `BeautifulSoup4` to extract structured profiles, which are then seeded into the database, as shown in Listing 1.

Listing 1: SQLAlchemy Faculty Schema Definition

```
class Faculty(Base):
    __tablename__ = "faculty"
    id = Column(Integer, primary_key=True)
    name = Column(String(255), nullable=False)
    clean_name = Column(String(255), nullable=False, index=True)
    designation = Column(String(255))
    biography = Column(Text)
    research_interests = Column(Text)
    contact_info = Column(String(255))
    profile_pdf = Column(String(512))
    source_url = Column(String(512))
    additional_info = Column(JSON)
```

4.2 Data Ingestion Pipeline

The unstructured database is populated by crawling the JUIT website (`www.juit.ac.in`) using HTTrack. The resulting HTML pages are chunked, parsed, and converted to embeddings using Ollama's `embeddinggemma:latest` model, then stored inside ChromaDB collections. High-priority documents, such as official university ordinances, are placed in a curated `vault/` directory and ingested directly.

5 Results, Evaluation & Discussion

The system was evaluated against a set of 150 test queries spanning faculty bios, fee regulations, chitchat, and random out-of-scope prompts.

Table 1: Performance Metrics of Luna Retrieval Tracks

Metric	**Chitchat (Router)**	**Track A+ (Official)**	**Track B (Website)**
Retrieval Precision	100%	98.6%	87.2%
Avg. Latency (ms)	4.2 ms	124 ms	845 ms
LLM Hallucinations	0%	0%	¡2.1%
Safety Valve Trigger	N/A	100% (Fuzzy Match)	N/A

As shown in Table 1, routing chitchat queries results in near-instant response times. Track A+ lookups execute quickly since SQL queries bypass embedding generations. For Track B, the hybrid search model maintains strong semantic coverage (87.2% precision) while the strict distance threshold successfully filters unrelated information.

6 Conclusion & Future Scope

In this project, we developed and deployed **Luna**, a hybrid Dual-Track RAG-based AI assistant for Jaypee University of Information Technology. By decoupling official records from crawled webpage knowledge, Luna maintains absolute factual accuracy for critical inquiries (fees, administration, faculty profiles) while offering broad coverage for supplementary campus information. The early-exit router and fuzzy-matching safety valves ensure computational efficiency and user-friendly query handling.

Future enhancements will focus on implementing automated daily scraping scripts to keep the local mirror in sync, integrating LangSmith for deep observability and tracing of token retrieval pathways, and introducing voice search functionality to assist visually impaired campus visitors.

References

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Lewis, M., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.

[2] Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333-389.

[3] Chroma Core Team. (2023). Chroma: The AI-native open-source embedding database. *GitHub Repository*. https://github.com/chroma-core/chroma.

[4] Ramírez, S. (2018). FastAPI: High performance, easy to learn, fast to code, ready for production. *GitHub Repository*. https://github.com/tiangolo/fastapi.

[5] Chase, H. (2022). LangChain: Building applications with LLMs through composability. *GitHub Repository*. https://github.com/langchain-ai/langchain.

[6] Bachmann, M. (2020). RapidFuzz: Rapid fuzzy string matching in Python using Levenshtein distance. *GitHub Repository*. https://github.com/maxbachmann/RapidFuzz.

[7] Gemma Team, Google. (2024). Gemma: Open models based on Gemini research and technology. *Technical Report, Google DeepMind*.